

Zápočtový program C# (dokumentace)

Adéla Volfová

Srpen 2025

Jako zápočtový projekt byla vypracovaná jednoduchá webová aplikace *Scheduly* sloužící k usnadnění základní organizace akcí pro více lidí. Projekt je založený na ASP.NET Core EF a Reactu. Více detailů níže v uživatelské příručce a programátorské dokumentaci.

1 Uživatelská dokumentace

Vstupním bodem aplikace je hlavní strana, kde je možné provést kliknutím na tlačítka Login či Sign up, přihlášení či registraci. Dále se na hlavní straně nachází možnost zadání kódu akce, kterým je možné se přihlásit k akci. Po zadání kódu je uživatel přesměrován na vyplnění jeho preferencí k akci. Po uložení bude přesměrován na detail akce, kde je možné vlastní preference upravovat, dokud není akce organizátorem uzavřena. Po přihlášení se uživatel může dostat pomocí navigační lišty na stranu "My dashboard", kde uvidí přehled všech akcí, kterých se účastní, tedy ke kterým se přihlásil pomocí kódu akce. A také akce, které sám organizuje. Tyto akce lze vytvořit tlačítkem "Add event". Při jeho stisknutí je uživateli zobrazen krátký formulář pro vyplnění základních údajů k akci jako je název a popis. Dále může uživatel zvolit typ akce, zda se bude jednat o akci "Open" či "Fixed" (viz nový odstavec). Při volbě "Open" akce uživatel zadává ještě hraniční termíny, tedy omezuje akci na daný časový úsek. Po vytvoření akce je uživatel automaticky zapsán jako její organizátor, což mu dodává určité schopnosti co se týče plánování akce. Na straně "My dashboard" je také možné, aby uživatel zrušil akci jako organizátor, stisknutím ikony koše, nebo odešel z akcí, kterých se účastní, stiskem ikony dveří.

1.1 Módy události

Systém podporuje dva základní módy události, které se liší v přístupu k výběru místa a času. Fixed Mode reprezentuje tradiční přístup, kde organizátor předem určí místo a čas události. Uživatelé mohou stále zadávat svoje preference a hlasuje se nad všemi variantami.

Open Mode je pokročilejší režim, kde se místo a čas určují dynamicky na základě analýzy uživatelských preferencí. Tento mód využívá plný potenciál

systému a umožňuje najít skutečně optimální řešení, které vyhovuje co nejvíce účastníkům.

1.1.1 Open event

Open akce je typ akce, který je z větší části zautomatizovaný. Organizátor při vytvoření akce pouze zadá limitní termíny. Akce je rozdělena do 3 fází. Proposal fáze, kdy program získává preference všech přibývajících účastníků. Do Final Voting fáze může organizátor přesunout akci stiskem tlačítka Finalize Proposals. Během tohoto přechodu program vybere nejvhodnější termín a 3 nejlepší lokality v závislosti na uložených hlasech. V této fázi si uživatelé vybírají již pouze z jedné možnosti. Do závěrečné fáze, opět stiskem tlačítka tentokrát Close Event, může organizátor posunout akci. Zde již na detailu akce uživatelé pouze uvidí pozvánku na akci s vybranou lokalitou a čas srazu.

1.1.2 Hlasování

Po vytvoření či přihlášení se k open event je ihned zobrazen formulář na vyplnění vlastních časových preferencí i preferovaného typu místa akce a lokality na mapě. Stisknutím tlačítka "Save preference" budou preference uloženy. Na stránce detailu akce, kam se lze dostat rozkliknutím akce na "My dashboard", bude vidět tlačítko Edit Preference, kterým je možné upravit své hlasy. Dále tu je možné vidět seznam účastníků akce a souhrnný graf časových možností všech účastníků. Organizátor má možnost na této stránce nastavit časovou náročnost akce v rozpětí 1 až 24 hodin.

1.1.3 Fixed event

Tento typ akce nabízí uživatelům jak pouze účastníkům, tak organizátorovi vybrat si vlastní místa i časy pro organizaci akce. Všichni mají neomezený počet hlasů pro jaké možnosti chtějí hlasovat. Své hlasy mohou upravovat dokud nebude akce ukončena organizátorem a bude zobrazena pozvánka s lokalitou a časem, který získal největší počet hlasů z předchozí fáze.

2 Programátorská dokumentace

2.1 Přehled projektu

Hlavní funkcionalitou je sbírání preferencí od uživatelů a následné automatické generování návrhů a integrace s Google Places API. Tento přístup umožňuje organizátorům najít optimální místa a časy pro setkání na základě skutečných potřeb a dostupnosti všech účastníků.

Aplikace funguje na principu postupného workflow, kde se nejprve sbírají preference uživatelů, následně se tyto data analyzují a generují se konkrétní návrhy, na které mohou uživatelé hlasovat. Celý proces je automatizovaný a poskytuje uživatelům intuitivní rozhraní pro správu událostí.

Frontend aplikace je implementován v Reactu s využitím přístupu založeného na komponentách. Pro stylování je použit Tailwind CSS, který umožňuje rychlé a flexibilní vytváření responzivního uživatelského rozhraní.

Aplikace je strukturována do dvou hlavních složek:

pages – obsahuje stránky aplikace, které reprezentují jednotlivé části uživatelského rozhraní, například dashboard, detail události nebo přihlašovací obrazovku,

components – obsahuje znovupoužitelné komponenty, jako jsou formuláře nebo karty událostí.

S backendem frontend komunikuje prostřednictvím knihovny Axios, která je využívána pro odesílání HTTP požadavků a získávání dat. Autentizace a autorizace uživatelů je řešena pomocí Auth Contextu, který spravuje stav přihlášení v rámci celé aplikace. Přihlašovací JWT token a userId jsou ukládány do Session Storage, aby byla zajištěna jejich dostupnost mezi jednotlivými stránkami i po obnovení aplikace.

Frontend byl navržen především za účelem demonstrování fungování backendové logiky a základní vizualizace dat. Z tohoto důvodu není doplněn o podrobnější dokumentaci

2.2 Backend - Technologická architektura

2.2.1 Použité technologie

Backend aplikace je postaven na ASP.NET Core Web API frameworku, který poskytuje platformu pro vytváření RESTful API služeb. Pro komunikaci s databází využívám Entity Framework Core ORM, které umožňuje práci s daty a automatické generování SQL dotazů. Databázový systém je navržen tak, aby podporoval jak SQL Server, tak PostgreSQL, což poskytuje flexibilitu při nasazení.

Autentifikaci jsem vyřešila pomocí JWT (JSON Web Token), což je moderní způsob, jak pracovat s přihlášením uživatelů. Pro bezpečné ukládání hesel používám knihovnu BCrypt, která se postará o jejich hashování. Díky tomu je autentifikace stateless, což je ideální pro webové API.

Pro zadávání preferovaných lokalit využívám OpenStreetMaps, což je bezplatná platforma pro získávání základních informací z map a pro získávání informací o místech využívám Google Places API, které poskytuje přesné a aktuální údaje o restauracích, kavárnách, parcích a dalších lokalitách. Tato integrace umožňuje systému nabídnout reálné a ověřené možnosti míst pro události.

2.2.2 Struktura projektu

Složka Controllers obsahuje API endpointy, které slouží jako vstupní bod pro HTTP požadavky a přeposílají business logiku na služby. Services vrstva obsahuje veškerou business logiku a je zodpovědná za zpracování požadavků, analýzu dat a komunikaci s externími API.

Models složka definuje databázové entity, které reprezentují doménové objekty aplikace. DTOs (Data Transfer Objects) jsou specializované třídy určené

pro přenos dat mezi frontend a backend vrstvami, přičemž zajišťují bezpečnost a zabráňují over-fetchingu dat.

Database složka obsahuje DbContext konfiguraci a definice vztahů mezi entitami. Data složka je zodpovědná za inicializaci databáze a seedování základních dat, jako jsou například základní role v aplikaci.

2.3 Databázové schema a entity

2.3.1 Základní entity

Entity Event představuje centrální prvek celého systému. Každá událost má jedinečný identifikátor, název a popis, které zadává organizátor. Vlastnost OwnerId odkazuje na uživatele, který událost vytvořil, což je důležité pro autorizaci operací. CreatedAt časová značka slouží pro budoucí případné řazení událostí.

Mode property určuje, zda se jedná o Fixed nebo Open událost, což ovlivňuje chování celého workflow. Phase sleduje aktuální stav události v rámci lifecycle. TimeRangeFrom a TimeRangeTo definují časové okno, ve kterém se může událost konat - tyto properties jsou relevantní pro Open Mode události.

Feature Code představuje prvních 6 znaků z GUID identifikátoru události. Tento kód slouží pro snadné sdílení události mezi uživateli bez nutnosti kopírování dlouhých GUID hodnot.

Final properties (FinalPlaceName, FinalAddress, FinalTimeFrom, FinalTimeTo) uchovávají konečné výsledky po ukončení hlasování. Tyto hodnoty se nastavují až po dokončení celého workflow.

2.3.2 Uživatelé a autentifikace

User entita obsahuje základní identifikační údaje jako Name a Email, přičemž Email slouží také jako uživatelské jméno pro přihlášení. PasswordHash ukládá BCrypt hash hesla s automaticky generovanou solí, což zajišťuje vysokou úroveň bezpečnosti.

Plaintext hesla se nikdy neukládají a nejsou dostupná ani administrátorům systému. Pro reset hesla by bylo nutné implementovat separátní workflow s temporary tokens.

Role systém implementuje hierarchii oprávnění. System roles (Admin, User) určují globální oprávnění v aplikaci, zatímco Event roles (Organizator, Participant) se vztahují ke konkrétním událostem. Tento přístup umožňuje flexibilní správu oprávnění. Prostor pro rozšíření z pohledu Admina.

2.3.3 Systém preferencí

PlacePreference entita je klíčovou součástí inteligentního doporučovacího systému. Každá preference obsahuje Type (typ místa jako kavárna, restaurace, park + možné rozšíření), geografické souřadnice (Latitude, Longitude) preferované lokace a seznam dostupných časových intervalů.

TimeInterval entity reprezentují konkrétní časové sloty, kdy je uživatel dostupný. Tyto intervaly jsou propojené s preferencemi a umožňují systému analyzovat překrývající se časy mezi všemi uživateli.

2.3.4 Hlasovací systém

Systém implementuje dva typy hlasování podle fáze události.

EventOptions reprezentují manuálně vytvořené možnosti pro hlasování. To je ukládáno pomocí Vote entity do databáze. Vote entita nyní neumožňuje uživatelům ohodnotit jednotlivé možnosti pomocí numerického skóre. Tato property je zde připravena pro budoucí rozšiřitelnost a možnost uživatelů k vyjádření větší náklonnosti k některým z možností. Vyšší skóre znamená vyšší preferenci, což umožňuje jemnější hodnocení než pouhé ano/ne hlasování.

GeneratedPlaceOptions jsou automaticky generované návrhy založené na analýze uživatelských preferencí. Tyto entity obsahují detailní informace o místě včetně GPS souřadnic, které získáváme z Google Places API. Navržený čas (TimeFrom, TimeTo) je vypočítán algoritmem pro optimalizaci překrytí dostupností uživatelů.

2.4 Vrstva služeb a business logika

2.4.1 TokenService a autentifikace

TokenService je zodpovědný za vytváření a validaci JWT tokenů. Při vytvoření tokenu se do claims ukládají NameIdentifier (user ID), Name (user name) a Role claims. Tyto informace jsou dostupné ve všech API endpointech bez nutnosti databázových dotazů, což zlepšuje performanci. Role claim umožňuje implementaci role-based authorization. Token je podepsán pomocí HMAC SHA512 algoritmu s klíčem načteným z konfigurace aplikace.

Doba platnosti tokenu je nastavena na 24 hodin, což představuje kompromis mezi bezpečností a uživatelským pohodlím. Po vypršení tokenu se uživatel musí znovu přihlásit. Service je registrován jako Scoped, což znamená, že jedna instance slouží po celou dobu HTTP požadavku.

2.4.2 GooglePlacesService a integrace s externími API

GooglePlacesService zapouzdřuje komunikaci s Google Places API a přijímá parametry jako typ místa, GPS souřadnice a požadované časové období.

Interně service sestavuje URL pro Google Places API s parametry pro nearby search. Používám rankby=distance, což zajišťuje, že dostaneme nejbližší místa seřazená podle vzdálenosti. API odpověď se deserializuje z JSON formátu do našich doménových objektů.

Integrace s Google Places API je implementována přes HttpClient, který je automaticky spravovaný pomocí AddHttpClient<T>() extension metody. Toto řešení zajišťuje proper disposal of connections a zabraňuje socket exhaustion problémům.

Google Places API má nastavené omezení počtu požadavků (rate limiting), které je řízeno pomocí kvót a omezení spojených s API klíčem. V aplikaci využívám pouze bezplatný tarif, který umožňuje posílat maximálně několik tisíc požadavků měsíčně. Po překročení tohoto limitu by bylo potřeba přejít na placený plán.

Service implementuje graceful error handling - pokud API volání selže, vrací prázdný seznam místo vyhození výjimky. Toto chování umožňuje aplikaci pokračovat v běhu i při dočasné nedostupnosti Google služeb. Výsledky jsou omezeny na 3 nejlepší možnosti.

2.4.3 EventService a core business logika

EventService představuje jádro aplikační logiky a řídí většinu business operací. CreateAsync metoda nejen vytváří novou událost, ale také automaticky přidává tvůrce jako organizátora do EventParticipants tabulky. Toto zajišťuje, že tvůrce má vždy odpovídající oprávnění ke své události.

GetByIdAsync metoda demonstruje pokročilé použití Entity Framework Include functionality pro eager loading souvisejících dat. Metoda načítá událost s účastníky a jejich uživatelskými daty v jednom databázovém dotazu, což optimalizuje výkon. Důležitá je také logika pro určení, zda je aktuální uživatel organizátorem události.

Nejsložitější metodou je FinalizeProposalPhase, která implementuje algoritmus pro analýzu preferencí a generování doporučení. Algoritmus začíná načtením všech preferencí pro danou událost včetně časových intervalů pomocí Include.

2.4.4 Algoritmus analýzy preferencí

Algoritmus pro analýzu preferencí funguje ve čtyřech hlavních krocích. Nejprve se určuje nejpopulárnější typ místa pomocí GroupBy operace na Type property všech preferencí. Typ s nejvyšším počtem výskytů se stává základem pro vyhledávání v Google Places API.

Druhým krokem je výpočet optimální lokace. Používám jednoduchý průměr zeměpisných šířek a délek, což funguje dobře pro menší geografické oblasti. Pro větší vzdálenosti by bylo potřeba implementovat pokročilejší výpočty.

Dále probíhá analýza časových preferencí. Algoritmus prochází všechny časové intervaly všech uživatelů a pro každou hodinu v týdnu počítá, kolik uživatelů je v tu dobu dostupných. Výsledkem je Dictionary, kde klíčem je DateTime reprezentující konkrétní hodinu a hodnotou je počet dostupných uživatelů.

Hodina s nejvyšším počtem dostupných uživatelů se stává základem pro naplánování události. Kolem této hodiny se vysadí požadovaná délka události, takže pokud je nejlepší hodina 15:00 a událost má trvat 2 hodiny, naplánuje se od 14:00 do 16:00.

Nakonec se volá GooglePlacesService pro získání konkrétních návrhů míst a výsledky se ukládají do databáze jako GeneratedPlaceOptions.

2.5 Konfigurace databáze a Entity Framework

2.5.1 UTC DateTime handling

Jednou z klíčových konfigurací je automatické převádění všech DateTime hodnot na UTC. Toto řešení eliminuje problémy s časovými pásmy, které jsou časté při práci s globálními aplikacemi. Při ukládání do databáze se všechny DateTime hodnoty automaticky převedou na UTC pomocí `ToUniversalTime()` metody.

Při čtení z databáze se hodnoty označí jako UTC pomocí `DateTime.SpecifyKind()`, což zajišťuje správnou interpretaci dat. Tato konfigurace se aplikuje automaticky na všechny DateTime a DateTime? properties všech entit pomocí reflexe v `OnModelCreating` metodě.

2.5.2 Composite keys a relationships

`EventParticipant` entita používá kompozitní primární klíč složený z `UserId` a `EventId`. Toto řešení implementuje many-to-many vztah mezi `Users` a `Events` s možností přidání dodatečných dat (jako je `Role`).

`Relationship` konfigurace definuje, jak Entity Framework mapuje vztahy mezi entitami. One-to-many vztahy jako `Event-Participants` nebo `PlacePreference-TimeIntervals` jsou konfigurované pomocí `HasOne/WithMany` pattern. Many-to-many vztah `User-Event` je implementován explicitně přes `EventParticipant` join entitu.

`Location` je konfigurován jako "owned type", což znamená, že se neukládá do samostatné tabulky, ale jako součást `GeneratedPlaceOption` entity. To je efektivnější pro jednoduché value objekty.

2.6 Inicializace databáze a migrace

`DbInitializer` třída zajišťuje, že databáze je při startu aplikace ve správném stavu. `MigrateAsync()` metoda aplikuje všechny pending migrace, což umožňuje automatické updates databázového schématu při nasazení nové verze aplikace.

Seeding procesní vytváří základní data potřebná pro běh aplikace. Role seeding zajišťuje, že v databázi existují všechny potřebné role (`Admin`, `User`, `Organizator`, `Participant`). Bez těchto rolí by nefungovala autorizace.

Default admin user creation je praktická funkcionality pro development a initial deployment. V současné implementaci jsou credentials hardcoded. Je důležité tyto údaje přesunout do konfigurace nebo environment variables.

2.7 Testování

Testování aplikace probíhalo postupně a bylo zaměřeno především na ověřování funkčnosti prostřednictvím frontendu. Během vývoje jsem aplikaci průběžně zkoušela a doplňovala nové funkce, které jsem následně testovala v uživatelském rozhraní.

Unit testy jsem vytvořila pouze pro jednu metodu, u které bylo potřeba podrobně ověřit její správné fungování. Ostatní části aplikace byly testovány manuálně při samotném vývoji.